Affordance-based Robot Manipulation with Flow Matching

Fan Zhang and Michael Gienger
Honda Research Institute EU
Email: firstname.lastname@honda-ri.de

arXiv:2409.01083v5 [cs.RO] 7 Nov 2025

***Abstract*—We present a framework for assistive robot manipulation that addresses two fundamental challenges: efficient adaptation of large-scale models for scene affordance understanding and effective learning of robot actions by grounding the visual affordance. To tackle the first challenge, we adopt a parameter-efficient prompt tuning method, prepending learnable text prompts to a frozen vision model to predict affordances, while considering spatial and semantic relationships in multi-task scenarios. For the second challenge, we propose a flow matching method, representing a robot visuomotor policy as a conditional process of flowing random waypoints to desired robot actions. We introduce a real-world dataset with 10 tasks to evaluate our approach. Experiments show our prompt tuning method achieves competitive or superior performance to other finetuning protocols across data scales, while satisfying parameter efficiency. Flow matching yields more stable training and faster inference, while maintaining comparable generalization performance to diffusion policy. Our framework seamlessly unifies parameter-efficient affordance learning and robot action generation with flow matching. https://hri-eu.github.io/flow-matching-policy/**

***Index Terms*—Visual Learning, Flow Matching, Affordance Learning, Human-Centered Robotics**

I. Introduction

RECENT advances in vision-language models (VLMs) present unprecedented opportunities to solve robot manipulation problems. Attempts in the field have focused on three primary aspects: 1) End-to-end learning manipulation from scratch. These approaches [48] make the fewest assumptions on tasks and are formulated in language-image-to-action prediction models. 2) Off-the-shelf-vision-language models for robot manipulation. These works have prompted pre-trained VLMs in various contexts of robot motion learning, including reward design for reinforcement learning [41], python coding [33], joint actions [67], etc. 3) Intermediate substrate to bridge high-level language-image instructions and low-level robot policies. These works usually introduce some form of prior derived from human knowledge as an intermediate stage to alleviate the sample inefficiency problem of end-to-end learning, including affordances [27], primitive skills [28], etc. In this paper, we follow the third line of work to unify a parameter-efficient affordance model and a low-level robot flow matching policy.

Extracting affordance knowledge has long inspired the robot community [68]. Recent state-of-the-art works have proposed the affordance-based robot policy by describing affordances with 6D poses or 2D point tracks [46, 3]. For example, [46] represents affordances as the relative poses between the robot end effector and the object at key stages of the task. [3] uses 2D keypoint track predictions to infer a sequence of rigid transforms of the object affordance to be manipulated. Other research decouples affordance learning and robot policy by using more straightforward affordance representations like 2D masks or keypoints [63, 70]. These works typically localize task-specific affordances directly on objects relevant to certain actions, given detailed text instructions. Our affordance model additionally considers the relational context needed for interactions, including the spatial and semantic functional interplay relationship between multiple elements. We concentrate on multi-task scenarios with text prompting. As shown in Fig. 1, given the same visual scene but with different language instructions, we aim to extract different affordances for robot policy learning through our proposed model. For example, given a simple instruction of 'feeding the human', a sequence of 2D affordance heatmaps on a spoon handle, food and a person's mouth are identified.

Recent affordance models have successfully fine-tuned pre-trained vision-language foundation models to extract affordance knowledge [52, 70]. To leverage the capabilities of pre-trained foundation models while simultaneously mitigating the associated computational costs, some works have explored parameter-efficient fine-tuning (PEFT) large vision-language models [29]. One representative line of PEFT research has concentrated on prompt tuning methods, which prepend learnable prompts to a large frozen pre-trained model and optimize them via gradients during finetuning. In contrast to studies in NLP, it has been shown that prompt tuning could match or even outperform full fine-tuning and adapter-based methods, but with substantially less parameter storage in visual domains [76]. Inspired in part by the notion of human cognitive penetrability mechanism [42] that uses linguistic knowledge to tune ongoing visual processing, we aim to incorporate learnable text-conditioned prompts into any vision foundation model while keeping it frozen, preserving its visual understanding capabilities, to learn instruction-relevant manipulation affordance maps.

The subsequent challenge involves deploying the visual affordance across robot manipulation learning paradigms. A recent line of work builds on successes in curriculum-based imitation learning [5], probabilistic movement primitive modeling [69], and diffusion models [7] to generate motion trajectories to capture multimodal action distributions. Flow Matching is another novel generative method. Sharing theoretical similarities with diffusion process, flow matching aims to regress onto a deterministic vector field to flow samples toward the target distribution. [34] has proven that the simplicity of flow matching objectives allows favorable performance in stable training and generation quality compared to solving complex stochastic differential equations in the stochastic denoising diffusion process. We extend flow matching to the robotics domain. As shown in Fig. 1, the proposed method would flow the random waypoints to the desired action trajectories based

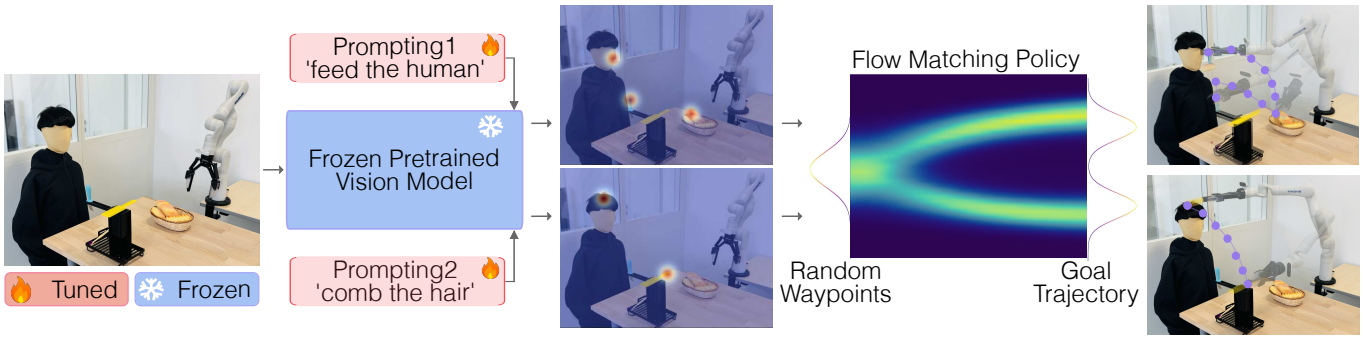


Fig. 1: The proposed framework of unifying affordance map learning and action generation for robot manipulation. Given the same visual scene with different language instructions, the model first extracts instruction-relevant manipulation affordances. This is achieved through a prompt tuning method that prepends learnable text-conditioned prompts in a frozen vision foundation model. Then, a flow matching policy is proposed to transform the random waypoints to the desired action trajectories, guided by task-relevant affordance maps.

on multi-task affordances in a single flow matching policy.

We also construct a Real-world Activities of Daily Living (ADLs) dataset with 10 tasks. The novelty of our dataset is that it contains the same scenarios but with multi-task affordance and robot trajectories. Experimental evaluation on our dataset empirically demonstrates that the prompt tuning method for learning affordances achieves performance competitive, and sometimes beyond other finetuning protocols across data scales and vision-language fusion architectures.

This work seamlessly grounds VLM-based affordance with flow matching for real-world robot manipulation. We leverage the capability of the flow matching policy to represent multi-modal action distributions for learning accurate 6D robot actions, while 2D affordance maps readily provide sufficient guidance in shaping the policy. Depending on the system design, our approach can run at approximately 30 Hz (affordance learning: 18.340 ms; flow matching with one-step inference: 13.228 ms), making it suitable for closed-loop manipulation tasks. We have further systematically evaluated robot manipulation with flow matching on several benchmarks, including various input representations, robot control types, and manipulation tasks. We showcase that across several benchmarks, flow matching attains favorable performance in training stability, generation quality, and computational efficiency amongst competing methods of behavior cloning. Our code is publicly available.

The main contributions can be summarized as follows:

1) A parameter-efficient prompt tuning method for adapting pretrained vision foundation model conditioned on language instructions to learn manipulation affordances, incorporating the spatial and semantic interactions between elements in multi-task scenarios.
2) A novel formulation using flow matching for closed-loop 6D robot manipulation learning from various inputs, including visual affordances. Empirical and extensive results show that flow matching leads to consistently favorable results than some alternative behavior cloning methods in various manipulation tasks. This includes **more stable training and evaluation and noticeably faster inference, while maintaining comparable generalization performance to diffusion policy (DDIM, DDPM and score-based models)**.
3) A framework seamlessly unifies parameter-efficient affordance learning and robot action generation with flow matching. Experimental results, compared against end-to-end learning and off-the-shelf VLM-based trajectory generation frameworks, demonstrate that affordance representations provide consistent guidance for shaping manipulation policies.

II. Related Work

A. *Affordance Learning and Affordance-Based Robot Policy*

Humans rely heavily on affordances to perform day-to-day tasks across environments efficiently. The concept of affordance has been introduced in [16], referring to the ability to perform certain actions with objects in the context of a given scene. Several state-of-the-art works have adopted this concept and successfully proposed affordance-based robot policy by learning the affordance parameterization that could be amenable to deployment on robots, including 6D pose [46], stable object placement [71], point tracks [3], and post-contact trajectory [2]. Other approaches decouple actionable representation for affordances from robot policy by using more straightforward affordance representations, like 2D masks [63], keypoints [44], heatmaps [73], part segmentation [9], dense image feature descriptors [12]. We follow this idea of decoupling and use 2D heatmaps as affordances.

Recent SoA works have successfully leveraged pre-trained vision-language models (e. g., LLaVA and Vicuna) to extract affordance knowledge, as in AffordanceLLM [52] and Robopoint [70]. This raises three important research questions: i) How can language-vision inputs for instruction-aware vision encoding be readily fused? ii) How to parameter-efficiently finetune a pretrained foundation model? and iii) How to encode the spatial and semantic functional interplay relationship between elements in multi-task scenarios for robot manipulation affordance learning? We introduce a prompt tuning method as a solution to these questions.

B. Parameter-Efficient Finetuning

Given the dominance of large-scale vision-language models, many approaches have been proposed to efficiently finetune a frozen pretrained model for downstream tasks to speed up training and reduce memory [19, 53]. Two representative parameter-efficient finetuning methods are adapters and prompting. The adapter-based research designs the adapter variants that could add extra lightweight modules [15, 25, 39, 58]. Other work focuses on prompt tuning [38, 36], which treats learnable prompts as continuous vectors and computes their gradients with backpropagation during training. Studies on randomly generalised trainable prompts [29] for universal use or condition-admitted prompt variables [59] for better specific task performance have both been explored. The extension of prompt tuning to vision tasks has gained massive success. Visual prompt tuning [29] has manipulated visual prompts to steer models in arbitrary vision tasks. Dancemvp [74, 75] has adjusted language prompts to guide text-audio models in performing dancing assessment tasks. Inspired by its recent success, we extend the prompt tuning technologies to address the challenge of adapting large pretrained vision-language models to affordance learning for robot manipulation. The intuition is clear: if the model understands the posed text instruction and the inherent context, it should extract visual affordances that directly correspond to the relevant image aspects. Our method achieves the above goal by integrating learnable text-conditioned prompts into a large vision encoder, while keeping it frozen to preserve visual understanding capabilities. Besides, recent research on VLM for affordance-based robot policy typically localizes task-specific affordances directly on objects relevant to certain actions, which requires detailed language instructions about all task-related objects and actions. We aim to enable our affordance model to incorporate relational context essential for interactions, capturing both spatial and semantic-functional relationships among multiple objects and persons, where only simple prompts are required.

C. Robot Learning from Demonstration

Imitation learning has been a common paradigm for robots, which requires simulated or real-world demonstration data collection [31]. To improve data efficiency, extensive work has been proposed to learn robot policies on top of visual representations [35], such as keypoints or affordance heatmaps [35], instead of end-to-end raw images [17]. This paper concentrates on using affordances to guide low-level robot manipulation. In terms of network architectures for robot learning, prior works have successfully investigated convolutional networks [31], Transformers [62], generative adversarial networks [23], Energy-Based Models [11], etc. However, the collected data is usually expected to be non-convex and multi-modal due to the variability in human demonstrations. Recent works have addressed this problem by reformulating the robot policy as a generative process. Diffusion policy [7], including Denoising Diffusion Probabilistic Models (DDPM [24]), Denoising Diffusion Implicit Models (DDIM [60]) and score-based models [54], has emerged as a powerful class of generative models for behavior cloning

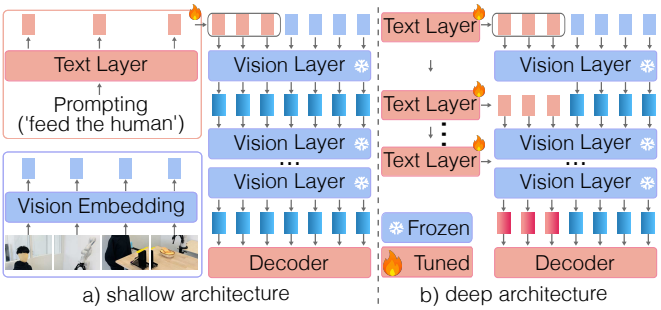


Fig. 2: Overview of prompt tuning structures used for affordance learning. (Left) For the shallow structure, text-conditioned prompts are prepended to the first vision transformer layer. (Right) For the deep structure, prompts are inserted into every vision layer. Only the prompt-related layers and the decoder are being updated during the training, while the vision transformer remains frozen.

by representing a robot's visuomotor policy as a conditional denoising diffusion process. In this work, we investigate flow matching [34], a novel generative model that has demonstrated its superiority in image generation, but is much less explored in robotics domains.

D. Flow Matching in Robotics

Despite its recent progress in image generation [1], the application of flow matching in robotics domains remains underexplored [26, 57]. Few prior studies have concentrated exclusively on certain robot scenarios for deploying flow matching, for pointcloud environment [8], dynamics [47], Riemannian manifolds [6], etc. We propose to use flow matching to learn multi-task robot behaviors from raw observations, including visual affordances obtained from a vision-language model, in a single supervised policy. We have first systematically evaluated robot manipulation with flow matching on several benchmarks, including various input representations, robot control types, and manipulation tasks.

III. Methods

A. Prompt Tuning for Affordance Map Learning

Providing any type of pre-trained vision transformer, our objective is to learn a set of text-conditioned prompts to maximize the likelihood of correct affordance labels, as shown in Fig. 2. Only the prompt-related layers and the decoder are being updated during the training, while the vision transformer remains frozen. Inspired by Vision Prompt Tuning [29], we propose two frameworks: shallow and deep network architectures.

1) Shallow Architecture: The vision transformer layer takes the image patch embeddings $\boldsymbol{E}_0$ as input and passes through various layers $\boldsymbol{L}_i^v$ to achieve vision features $\boldsymbol{E}_i$, where $\boldsymbol{E}_i \in \mathbb{R}^{M \times C}$ and C is the channel dimension.

$$\boldsymbol{E}_i = \boldsymbol{L}_i^v(\boldsymbol{E}_{i-1}) \qquad i = 1, 2, \cdots, N$$

Similarly, the text transformer layer could be represented as

$$\boldsymbol{P}_i = \boldsymbol{L}_i^p(\boldsymbol{P}_{i-1}) \qquad i = 1, 2, \cdots, N$$

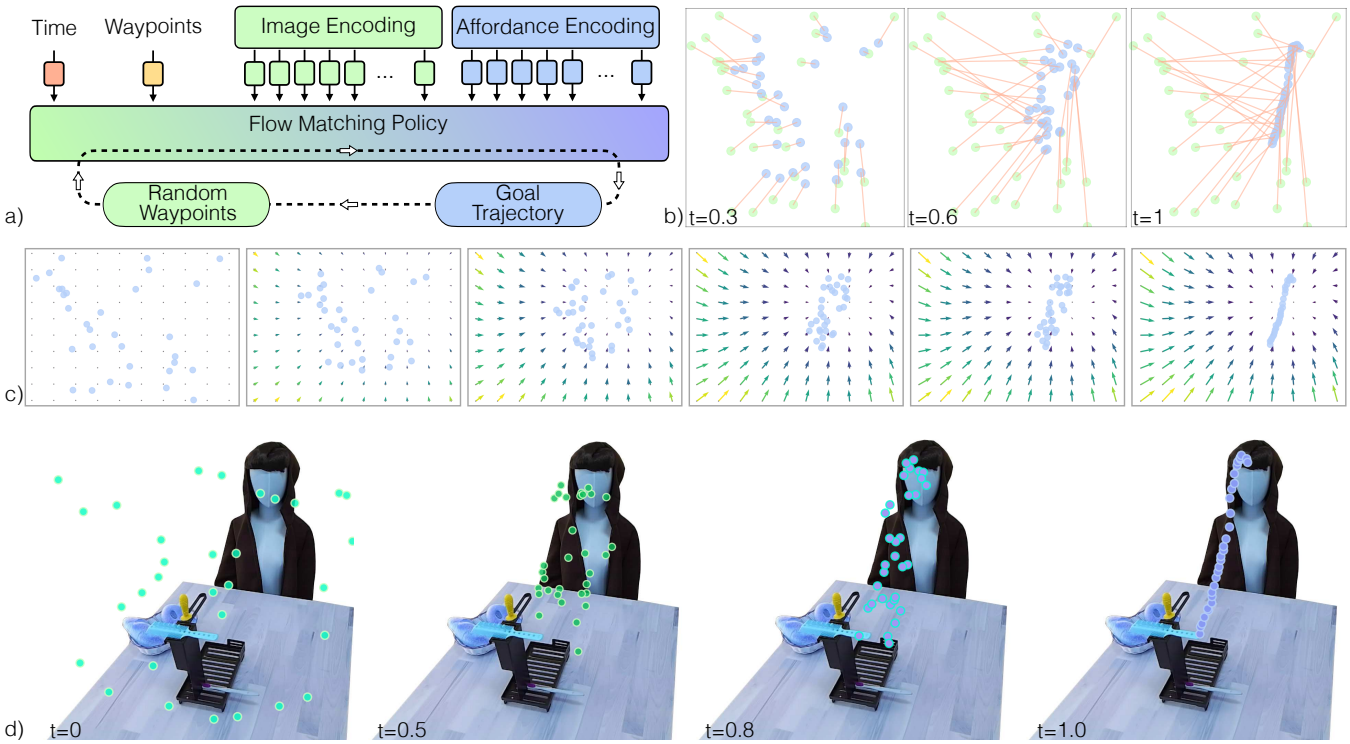


Fig. 3: Framework of flow matching policy. (a) General formulation. At each time step, flow matching takes visual observation $\boldsymbol{o}$ (e. g., state-based inputs, RGB-D images, visual affordances) as input, and outputs robot actions (e. g., 6D robot end-effector trajectories, robot joint actions, gripper actions). (b-d) Visualization of the inference process of transforming random waypoints to target actions over time from 0 (green) to 1 (purple). Red lines in (b) denote the flow paths. Vector fields are shown in (c).

where $\boldsymbol{P}_0$ denotes the text tokens, text features $\boldsymbol{P}_i$ are obtained through various layers $\boldsymbol{L}_i^P$, where $\boldsymbol{p}_i \in \mathbb{R}^{K \times C}$.

As shown in Fig. 2, for the shallow structure, only one text transformer layer is used to compute text features $\boldsymbol{P}_1$, which are then treated as prompts and inserted into the first vision transformer Layer:

$$[\boldsymbol{Z}_1, \boldsymbol{E}_1] = \boldsymbol{L}_1^v([\boldsymbol{P}_1, \boldsymbol{E}_0])$$
$$[\boldsymbol{Z}_i, \boldsymbol{E}_i] = \boldsymbol{L}_i^v([\boldsymbol{Z}_{i-1}, \boldsymbol{E}_{i-1}])$$

Then a decoder is added on the global output flattened token sequence to generate visual affordance tokens.

$$\text{Affordance} = \text{Decoder}(\boldsymbol{Z}_N, \boldsymbol{E}_N)$$

2) Deep Architecture: For the deep architecture, the only difference is that text features $\boldsymbol{P}_i$ are computed through each layer and introduced at the corresponding vision transformer layer's input space:

$$[_, \boldsymbol{E}_1] = \boldsymbol{L}_1^v([\boldsymbol{P}_1, \boldsymbol{E}_0])$$
$$[_, \boldsymbol{E}_i] = \boldsymbol{L}_i^v([\boldsymbol{P}_i, \boldsymbol{E}_{i-1}])$$

This is different to parallel adapters, which introduce additional trainable modules that run in parallel with the main transformer layers instead of modifying existing layers.

3) Implementation Details: In our implementation, we select and adopt a single structure from the two designs. The deep structure demonstrates better performance than the shallow structure, albeit with increased computational cost. Our goal is to integrate textual representations into any vision encoder while keeping it frozen, preserving its visual understanding capabilities. Thus, we have chosen the most basic vision backbone, a pretrained ViT-B-16 transformer. We use the classic CLIP Transformer layers [53] that output 76 tokens. As suggested by MAE [22], the decoder is only used for downstream tasks and could be flexible and lightweight. Thus we use one single transformer decoder layer.

We use the L2 Mean Squared loss between the predicted and ground truth affordances for network training. The training parameters for the prompt tuning network include an image size of 224 × 224, AdamW with a learning rate of 1.5e-4, including Warmup with step-decay, and a batch size of 256. We add positional embeddings to all the image and language tokens to preserve the positional information. In the subsequent experiments, we will further ablate multiple model variants, including text and vision fusion structures, prompt depth, pretrained weights for vision transformers, etc.

B. Flow Matching Policy

We build the robot behavioral cloning policy as a generative process of flow matching, which constructs a flow vector that continuously transforms a source probability distribution toward a destination distribution. Flow matching leverages an ordinary differential equation to deterministically mold data distribution, contrasting with Denoising Diffusion Probabilistic Models (DDPM), which is based on a stochastic differential equation by introducing noise.

Algorithm 1 Robot Flow Matching Policy Training

Input: observation $\boldsymbol{o}$, target robot actions $\boldsymbol{x}_1$, source random waypoints p_0
Output: flow $\mathbf{v}_{\boldsymbol{\theta}}$

```
while not converged do
    x_0 ~ p_0, sample random robot waypoints
    t ~ U[0, 1], sample time steps
    x_t = t x_1 + (1 − t) x_0, linear interpolation
    v_t(x|o) = v_θ(x_t, t|o), flow estimation
    ∇_θ ‖v_θ(x_t, t|o) − ẋ_t‖, gradient step
end while
Stopping criteria: training epochs reached
```

Algorithm 2 Robot Diffusion Policy (DDPM) Training

Input: observation $\boldsymbol{o}$, target robot actions $\boldsymbol{x}_1$, source Gaussian noises p_0
Output: noise $\epsilon_{\boldsymbol{\theta}}$

```
while not converged do
    x_0 ~ p_0, sample Gaussian noises
    t ~ U[0, 1], sample time steps
    x_t = N(x_t; √ᾱ_t x_0, (1 − ᾱ_t)I), forward process
    ε_t(x|o) = ε_θ(x_t, t|o), noise estimation
    ∇_θ ‖ε_θ(x_t, t|o) − ε_t‖, gradient step
end while
Stopping criteria: training epochs reached
```

1) Flow Matching Model: Given a conditional probability density path $p_t(\boldsymbol{x}|\boldsymbol{z})$ and a corresponding conditional vector field $\boldsymbol{u}_t(\boldsymbol{x}|\boldsymbol{z})$, the objective loss of flow matching could be described as:

$$\mathcal{L}_{\text{FM}}(\boldsymbol{\theta}) = \mathbb{E}_{t,q(\boldsymbol{z}),p_t(\boldsymbol{x}|\boldsymbol{z})} \left\|\mathbf{v}_t(\boldsymbol{x},\boldsymbol{\theta}) - \mathbf{u}_t(\boldsymbol{x}|\boldsymbol{z})\right\|^2 \quad (1)$$

where $\boldsymbol{x} \sim p_t(\boldsymbol{x}|\boldsymbol{z})$, $t \sim \mathcal{U}[0,1]$. Flow matching aims to regress $\mathbf{u}_t(\boldsymbol{x}|\boldsymbol{z})$ with a time-dependent vector field of flow $\mathbf{v}_t(\boldsymbol{x},\boldsymbol{\theta})$ parameterized as a neural network with weights $\boldsymbol{\theta}$. $\mathbf{u}_t(\boldsymbol{x}|\boldsymbol{z})$ can be further simplified as:

$$\mathbf{u}_t(\boldsymbol{x}|\boldsymbol{z}) = \boldsymbol{x}_1 - \boldsymbol{x}_0 \qquad \boldsymbol{x}_0 \sim p_0, \boldsymbol{x}_1 \sim p_1$$

p_0 represents a simple base density at time $t = 0$, p_1 denotes the target complicated distribution at time $t = 1$, $\boldsymbol{x}_0$ and $\boldsymbol{x}_1$ are the corresponding samplings. $\mathbf{v}_t(\boldsymbol{x},\boldsymbol{\theta})$ is described as:

$$\mathbf{v}_t(\boldsymbol{x},\boldsymbol{\theta}) = v_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t) \quad (2)$$

where we define $\boldsymbol{x}_t$ as the linear interpolation between $\boldsymbol{x}_0$ and $\boldsymbol{x}_1$ with respect to time $\boldsymbol{x}_t = t\boldsymbol{x}_1 + (1-t)\boldsymbol{x}_0$, following the linear conditional flow theory [50]. $v_{\boldsymbol{\theta}}$ is a network of the flow model. Thus Equation (1) could be reformatted as

$$\mathcal{L}_{\text{FM}}(\boldsymbol{\theta}) = \mathbb{E}_{t,\sim p_0,\sim p_1} \left\|v_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t) - (\boldsymbol{x}_1 - \boldsymbol{x}_0))\right\|^2 \quad (3)$$

This represents the progression of the scalar flow that transforms data from source to target between time 0 and 1.

2) Flow Matching for Visuomotor Policy Learning: We extend flow matching to learn robot visuomotor policies. This requires two modifications in the formulation: i) modeling the flow estimation conditioned on input observations $\boldsymbol{o}$; ii) changing the output $\boldsymbol{x}$ to represent robot actions. Fig. 3 illustrates our model structures.

Visual observation Conditioning: We modify Equation (2) to allow the model to predict actions conditioned on observations:

$$\mathbf{v}_t(\boldsymbol{x}|\boldsymbol{o}) = v_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t|\boldsymbol{o})$$

Closed-loop action trajectory prediction: We execute the action trajectory prediction obtained by our flow matching model for a fixed duration before replanning. At each step, the policy takes the observation data $\boldsymbol{o}$ as input and predicts Tp steps of actions, of which Ta steps of actions are executed on the robot without re-planning. Tp is the action prediction horizon and Ta is the action execution horizon. The whole training process of flow matching is illustrated in Algorithm 1.

Inference: For the inference procedure, random waypoints are sampled from the source distribution and then flowed into the target trajectory by estimating the flow from $t = 0$ to $t = 1$ over steps. We could use multiple steps $1/\Delta t$ for inference:

$$\boldsymbol{x}_{t+\Delta t} = \boldsymbol{x}_t + \Delta t f(\boldsymbol{x}_t, t|\boldsymbol{o}), \qquad \text{for } t \in [0,1] \quad (4)$$

3) Implementation Details: For the network structures of flow matching, we first use ResNet [21] for visual embeddings $\boldsymbol{o}$. The flow model $f_{\boldsymbol{\theta}}$ is represented with U-Net [56]. The flow model predicts vectors $\mathbf{v}_t$ conditioned on visual observation embeddings $\boldsymbol{o}$ with Feature-wise Linear Modulation (FiLM) [49] as well as the interpolated waypoints $\boldsymbol{x}_t$. In the subsequent experiments, we will further study multiple model variants, including a transformer-based structure, trajectory representation.

In our case of robot manipulation, $\boldsymbol{x}_1$ in Equation (3) represents the demonstration robot action trajectories. $\boldsymbol{x}_0$ is the random generated waypoints following a multivariate normal distribution $\boldsymbol{x}_0 \sim \mathcal{N}(0, I)$. $\boldsymbol{x}$ here could denote 6D robot end-effector trajectories, robot joint actions, gripper actions, etc. The visual embeddings $\boldsymbol{o}$ include various types of inputs, such as state-based inputs, RGB-D images, and visual affordances.

4) Comparisons against Diffusion Policy: In this section, we provide some insights and intuitions about flow matching and its comparisons against diffusion policy for clarification. Algorithm 1 and Algorithm 2 respectively shows the pseudocode of training flow matching and diffusion policy with DDPM. We can see several similarities and differences between these two methods:

- **Sampling**: By solving a Stochastic Differential Equation (SDE), DDPM generates a clean sample from Gaussian noise. Flow matching regresses onto a target flow vector field that generates a deterministic mapping from source to target data distributions by solving an ordinary differential equation (ODE), which drops all the Gaussian assumptions.
- **Probability path**: Flow matching includes a particularly interesting family of probability paths: the conditional vector field with linear interpolant. Flow matching paths with linear interpolation are simpler than diffusion paths, forming straighter trajectories, whereas diffusion paths result in curved paths. These properties seem to empirically translate to more stable training, faster generation, and better performance.
- **Reparameterization**: [40, 45] claims that diffusion models and flow matching can be considered equivalent with repa-

rameterization, including rescaling time and reparameterizing the marginal velocity field via the score function. Specifically, [45] argues that DDIM is equivalent to flow-matching with the following parameters:

$$\boldsymbol{v}_t^{[\boldsymbol{x}_1,\boldsymbol{x}_0]}(\boldsymbol{x}_t) := \frac{1}{2t}(\boldsymbol{x}_t - \boldsymbol{x}_0) \qquad \text{DDIM}$$

$$\boldsymbol{v}_t^{[\boldsymbol{x}_1,\boldsymbol{x}_0]}(\boldsymbol{x}_t) := \frac{1}{2\sqrt{t}}(\boldsymbol{x}_0 - \boldsymbol{x}_1) \qquad \text{Flow Matching}$$

can both generate the same trajectory:

$$\boldsymbol{x}_t = \boldsymbol{x}_0 + (\boldsymbol{x}_1 - \boldsymbol{x}_0)\sqrt{t}$$

That being said, DDIM at time t corresponds to flow matching at time $2\sqrt{t}$; thus, flow matching is "slower" than DDIM when t is small. This may be beneficial for flow matching in practice.

C. Activities of Daily Living Dataset

We construct a real-world dataset with 10 tasks across Activities of Daily Living. Each task includes $1,000$ sets of RGB-D images, demonstrated robot action trajectories, and labeled ground truth of affordance maps. Thus $10,000$ demonstrations have been collected in total. The data has been manually collected by moving robot end-effectors with kinesthetic teaching. The novelty of our dataset includes: (i) It contains the same scenarios with multiple objects, multi-task affordances, and demonstrated robot trajectories. (ii) All tasks are related to Activities of Daily Living that involve (simulated) human data.

We label the affordance heatmaps with 2D Gaussian blobs centered on the object pixels of the demonstrated action. The affordance maps model the locations of all relevant object areas that physically interact with robots, given each task. For example, the feeding task requires affordance heatmaps centered on the fork handle, food, and the human mouth.

Objects are randomly placed within the range of the table (1.5 meters $\times$ 1 meter). The human (manikin) is placed randomly around the table in the camera view. We have around 30 different objects. Our tasks include prompt primitives: 'sweep the trash', 'pass the water to the human', 'hang the towel', 'put on the hat', 'cover the food', 'wipe the nose', 'wipe the forearm', 'feed the human', 'comb the hair', and 'brush the teeth'. We can also optionally add a pretrained LLM layer (e. g., GPT) in the very front for zero-shot text classification, allowing for linking other language instructions to one of the ten prompt primitives. For example, an ambiguous instruction 'I am hungry' can be linked to the prompt primitive 'feed the human'. The camera is positioned with some variations but generally oriented towards the table and objects. Please refer to the supplementary materials for more videos of each task.

IV. Experiments

In this section, we systematically evaluate the performance of the proposed prompt tuning and flow matching methods.

In Section IV-A, we first benchmark our proposed prompt-tuning structures against several commonly used parameter-efficient fine-tuning approaches to demonstrate the superior performance of prompt tuning. We then compare our method with state-of-the-art VLM-based affordance learning approaches to highlight its efficiency. Additionally, we perform ablation studies to analyze the impact of different design choices on performance. In Section IV-B and IV-C, we compare our flow-matching policy with state-of-the-art generative learning, imitation learning, and VLA models for robot manipulation through real-world experiments, evaluating performance in terms of generation quality, stability, inference time, and training resource requirements. In Section IV-D, we systematically investigate the performance of flow matching compared to diffusion policy, across simulation and real-world benchmarks.

A. Affordance Evaluation with Prompt Tuning

1) Baseline Studies: We benchmark our proposed prompt tuning structures against several commonly used parameter-efficient finetuning protocols and SoA VLM-based affordance learning methods:

- Full fine-tuning: fully update the text and vision transformer layers and the decoder.
- Adapter-based methods: insert MLP layers with residual connections between pretrained frozen transformer layers of vision and language, as customary in the literature [39, 58].
- Side-network methods: train a language-based network on the side, append pretrained vision features and sidetuned text features before being fed into the decoder, as customary in the literature [14]. This also shares similarities with parallel adaptors. Our deep prompt tuning method differs in the sense that it inserts text layers into every vision layer, while parallel network methods introduce additional trainable modules that run in parallel with the main transformer layers instead of modifying existing layers.
- Decoder-based methods: adopt the pretrained backbone as a feature extractor with fixed weights during tuning, and only train the decoder, as customary in the literature [21].
- Cross-attention methods: use cross-attention fusing text and vision instead of simple prepending. An example of cross-attention fusing vision and language can be found in the literature [30].
- Mixed-design methods: mixes the favorable adaptor designs. We compare against the state-of-the-art MAM adaptor [20], which is a Mix-And-Match adapter designed based on the practical findings on NLP.
- VLM-based affordance learning: We also compare against two state-of-the-art affordance learning methods based on fine-tuning large VLMs: AffordanceLLM [52] and Robopoint [70].

For a fair comparison, all the baselines here use self-supervised pretrained MAE weights on ImageNet-21k dataset for the vision transformer model. We randomly split our Real-world Activities of Daily Living (ADLs) dataset with 80%-20% percentage of training and testing. The results reported here are obtained after $1,000$ epochs of training.

2) Main Results: We use two metrics to evaluate our results: (i) L2 error of affordance heatmap estimation, and (ii) L2 distance between the predicted and ground truth of heatmap centers. We fit Gaussian Mixture Models on predicted heatmaps to determine the inferred heatmap centers. The heatmap error

	Methods	Learnable Params ↓	Affordance Heatmaps ($\times 10^{-3}$) ↓	Heatmap Centers (pixel) ↓
Baselines	Full	153.8M	**0.76**	**1.15**
	Decoder	3.9M	1.51	13.48
	Adapter	19.2M	1.17	6.22
	Cross-attention	43.5M	1.26	8.89
	side-network	42.7M	1.35	9.20
	mixed-design	108M	0.85	2.96
	AffordanceLLM	7B	**0.72**	**1.01**
	Robopoint	12B	-	3.18
Ours	PT-shallow	8.0M	1.42	12.04
	PT-deep (self-supervised weights)	42.1M	**0.80**	**2.93**
Ablations	PT-deep (supervised weights)	42.1M	1.48	10.13
	PT-deep (image output)	42.1M	1.56	13.27

TABLE I: Results of prompt tuning baseline and ablation studies. We report the number of learnable parameters, the heatmap estimation error (the fourth column) and the heatmap center error (the fifth column). Our method outshines other baselines except for the full finetuning.

is averaged on each map, and the center error is averaged on per center point. Three observations could be made:

- **General analysis**: Table I presents the results of prompt tuning on our ADLs testing dataset for affordance learning, comparing against baselines. The deep structure of prompt tuning outperforms other parameter-efficient baselines. We can also see that deep prompt tuning achieves better performance than RoboPoint but falls short of the performance achieved by AffordanceLLM. We would like to mention that AffordanceLLM and Robopoint are not parameter-efficient models (the focus of our research), as they involve finetuning large models, including LLaVA and Vicuna. Full finetuning such a large model may not function optimally if only a small dataset is available.
- **Prompt tuning against full finetuning**: Full finetuning slightly outperforms deep prompt tuning in terms of heatmap estimation error and heatmap center error. However, the distinction of heatmap center errors (1.78 pixels) remains subtle, given the full image size of 224×224. This outcome is favorable as it indicates that most heatmap errors are caused by the tails of the Gaussian distribution, instead of the center area where the robot actions are actually applied. We will further ablate the impact of dataset size on these two methods.
- **Generalizability**: We also observe that the trained model could be generalized to new objects. For example, the training dataset only includes a manikin. We found out that it generates well on our testing data with real humans. Affordances on objects with similar shapes (e. g., forks and spoons) could also be transferred. Note that as the proposed tuning method is parameter-efficient, it is envisaged that the method could be readily transferred to different tasks with a small amount of task-specific data. Note that the testing experiments involved only the authors as participants and were therefore exempt from the requirement for ethics approval.
- **What do prompts learn?** We show a t-SNE [65] visualization of the embeddings after the last vision transformer layer (before the decoder) in Fig. 4. We can see that the points of the same color (e. g., tasks with the same language prompts) are embedded together, implying that the representations recover the underlying manifold structure of discriminative task information.
- **Prompt tuning or adapters?** As pointed by the research of Visual Prompt Tuning [29], in contrast to comparable studies in NLP, prompt tuning outperforms full fine-tuning and adapter-based methods in the visual domain. The MAM adaptor [20] mixes the favorable adaptor designs based on the practical findings on NLP and achieves state-of-the-art results, but does not function optimally in the image-text domain.

3) Ablations: We further ablate model design choices:

Pretrained Weights: We evaluate using MAE self-supervised pretrained weights and supervised pretrained weights trained on ImageNet-21k dataset for the vision transformer model. The results in Table I show that self-supervised pretrained weights perform better. We are aware of other more complicated variants of vision transformers, for example, CLIP vision encoder and its pretrained weights. As our goal is to integrate textual representations into any vision encoder while keeping it frozen, we have chosen the most basic ViT-B-16 transformer backbone and commonly used pretrained weights and achieved competitive results.

Decoder Input: We apply the decoder on the global output and image-corresponding output after the vision transformer respectively and report results in Table I.

Dataset Size: We use various amounts of data for training. Fig. 5-left shows that prompt tuning has better adaptability than full finetuning when downstream data is scarce.

Prompt Location: We have seen different conclusions from prior works about whether the vision-language fusion should be integrated at early or late transformer layers. We conduct experiments to insert prompts at various layers. From Fig. 5-right, we can see that inserting prompts to early layers (for example, layer 1-3 from bottom to top) achieves higher loss than inserting to late layers (for example, layer 1-3 from top to bottom). Thus in our case, prompts have greater significance at the late transformer layers. These results are also supported by the nature of the vision transformer hierarchy: lower layers mainly capture low-level fundamental visual details, while higher layers focus on high-level concepts that might be vital for downstream tasks.

In conclusion, we observe no single method that outperforms all the rest. For scenarios where a small number of parameters or datasize is available, we reckon that prompt tuning remains the preferred approach.

B. Flow Matching Policy Evaluation

1) Baseline Studies: We compare our flow matching policy against: (i) Diffusion Policy [7] with DDPM and DDIM, (ii) Transformer-based behavior cloning, as customary in RVT [17], RT-X [48], and (iii) Score-based generative models [61] in Section IV-D.

Note that we are aware of other competitive robot behavior cloning methods, including energy-based IBC [11], GAIL [23], etc. Since extensive studies have been conducted and showed better performance of the diffusion policy against these methods, we choose the representative transformer and diffusion baselines for evaluation. Table V shows the hyperparameters used in flow matching and diffusion policy.

We first train and test flow matching and baselines on our collected Real-world Activities of Daily Living (ADLs) dataset

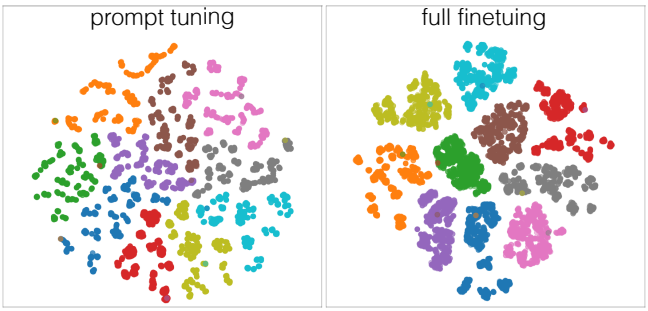


Fig. 4: t-SNE visualizations of the embeddings before the decoder. The points of the same color denote the tasks with same language prompts, which are embedded together. The prompt tuning method could produce instruction-relevant features without updating vision backbone parameters.

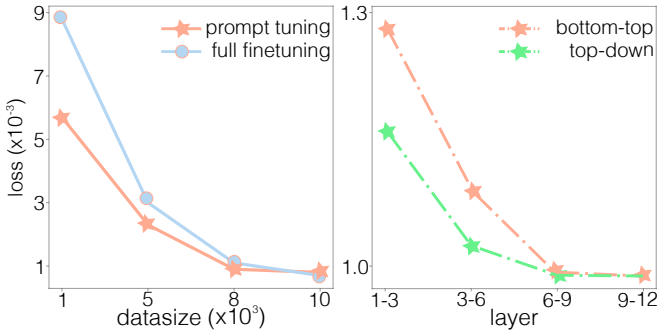


Fig. 5: Ablation studies of prompt tuning. We investigate the effect of various design choices on affordance learning performance, including pretrained weights, decoder input, dataset size and prompt location.

in a supervised manner with Mean Square Error Loss. In the following sections, we will also evaluate on other benchmarks (e. g., Push-T, Franka Kitchen, and Robomimic).

• **ADLs benchmark**: For 2D data, the training uses a RGB image with visual affordances as input, and the output is a trajectory in 2D pixel space. For the counterparts in 3D, the training takes the concatenation of RGB-D images with visual affordances as input, and outputs a trajectory in 3D Cartesian space. Since the ADLs task is not closed-loop, as shown in Table VI, we feed the image with all task-related object affordances to the flow matching policy, which directly outputs the entire trajectory. We randomly split the dataset with 80%-20% percentage of training and testing. The results are reported on the testing dataset, with each experiment conducted one time, after $1,000$ epochs of training. For each baseline, all 10 tasks in our dataset are trained in a single policy in a supervised manner. To ensure fair comparisons, the corresponding hyperparameters across the flow matching policy and baseline methods are selected to remain consistent. Table VI shows the task summary.

2) Main Results: Table II presents the results of flow matching policy on our ADLs testing dataset for robot trajectory learning, comparing against baselines. We use two metrics for evaluation: (i) the average error of 2D and 3D trajectory estimation, and (ii) the average inference time, performed with

	Methods	2D Trajectory Prediction (pixel) ↓	3D Trajectory Prediction (cm) ↓	Inference Times (ms) ↓
Ours	Flow Matching (Transformer, 16-step)	1.061	1.239	140.71
	Flow Matching (CNN, 16-step)	**0.840**	**1.009**	98.981
Ablations	Flow Matching (CNN, 1-step)	0.888	1.031	13.228
	Flow Matching (CNN, 4-step)	0.846	1.014	41.494
	Flow Matching (CNN, 8-step)	0.842	1.013	78.222
Baselines	DDPM (1-step)	2.851	2.479	13.791
	DDPM (4-step)	0.890	2.411	45.736
	DDPM (8-step)	0.884	2.403	80.200
	DDPM (16-step)	0.882	2.398	99.197
	DDIM (1-step)	4.328	10.78	13.757
	DDIM (4-step)	0.876	2.299	42.672
	DDIM (8-step)	0.875	2.272	80.940
	DDIM (16-step)	0.874	2.266	98.680
	Transformer-based BC	2.797	4.911	7.59

TABLE II: Results of flow matching policy against baselines and ablations. We report the average error of 2D and 3D trajectory estimation and the inference time. Our flow matching method achieves the best trajectory estimation accuracy. We also investigate the effect of various design choices on flow matching performance, including network structures, training, and inference steps.

RTX 4090 GPU. The trajectory error is averaged on each point of the trajectory.

Four observations could be made from this result:

• **Generation quality:** Flow matching (CNN-based, 16 steps) outperforms diffusion policy and Transformer baselines in terms of 2D and 3D trajectory prediction accuracy. The suboptimal performance of Transformer behavior cloning is expected as it is hindered by the nature of multi-modal action distribution, causing the averaging out across non-convex spaces.

• **Stability:** Fig. 6 shows an example of training and testing loss of flow matching and diffusion policy with DDPM throughout the training process. We can see **flow matching exhibits greater stability on both training and evaluation than diffusion policy**.

• **Inference time:** We have two observations here: i) Table II showcases that flow matching with 16 steps achieves faster inference time compared to diffusion policy with 16 steps. We hypothesize that flow matching with linear pointwise flows generates straighter flows than DDPM and DDIM, and thus causes faster inference. ii) More importantly, Table II also showcases that 1-step flow matching (error: 1.031cm, time: 13.228ms) has achieved comparable performance as 16-step DDIM (error: 2.266cm, time: 98.680ms), but **noticeably lowered inference time roughly by 85%**. We hypothesize that this is because diffusion models solve a stochastic differential equation with a series of discrete steps to progressively refine the generated sample. Contrarily, flow matching trains continuously normalize flow models, leading to no significant improvements when increasing inference steps. Thus 2-step flow matching has achieved comparable performance as the 16-step diffusion policy, which considerably reduces the inference time for closed-loop robot manipulation. This is **in line with the results obtained in the image generation domain**. As pointed out by Stable Diffusion 3 [10], flow matching outperforms diffusion policy with fewer inference steps, making it particularly advantageous in scenarios that demand fast inference.

• **Training resources:** DDPM training and benchmarking demand significant resources for various training and inference

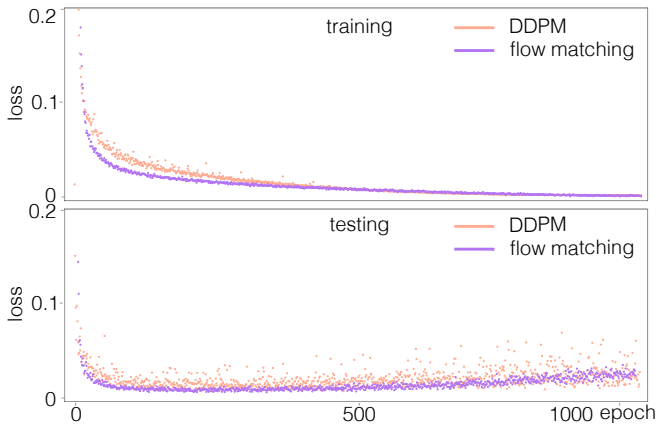


Fig. 6: Training and testing loss throughout the training process. Flow matching exhibits greater stability on both training and evaluation than diffusion policy.

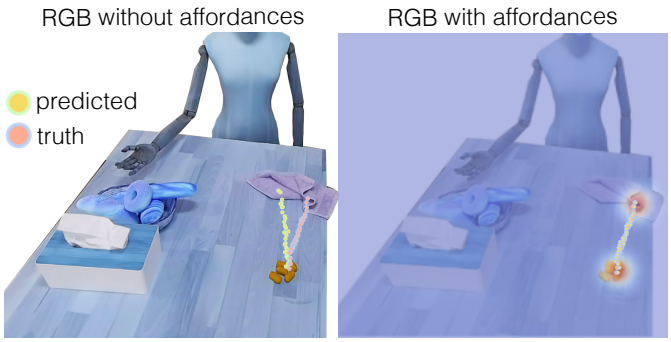


Fig. 7: Ablations of using RGB images with/without affordances for policy training. Visual affordance guides the flow matching policy to generate a trajectory closely aligned with the truth. The policy without affordances might generate a trajectory detached from the ground truth, but still a reasonable solution. This reinforces the argument that flow matching policy could handle multimodal robot action distributions.

steps. DDIM decouples the number of denoising iterations in training and inference, thereby allowing the algorithm to be trained one time with a large training iteration and use fewer iterations for inference to speed up the process. However, flow matching still achieves faster inference than DDIM.

3) Ablations: We further ablate policy design choices.

Network Structure: As shown in Table II, CNN-based flow matching achieves better results than transformer-based architecture. We hypothesize that transformer might need additional hyperparameter tuning.

Trajectory Representation: We empirically test trajectory representation with 8, 16, 32 and 64 waypoints. More waypoints are not necessary, while fewer waypoints are unable to entirely encapsulate the complete long-horizon trajectories. We have found that, in general, the trajectory representation does not wield a significant influence on flow matching performance. The performance reported in the above main results section is achieved by using 32 waypoints.

Methods (Inference Step)	**Flow Matching (16-step) ↑**	**Diffusion Policy (16-step) ↑**	**Transformer BC ↑**	**Flow Matching end-to-end ↑**
Activities of Daily Living	**0.82**	0.76	0.44	0.74

TABLE III: Real-world robot experimental results.

C. Real-World Robot Evaluation

We deploy our flow matching policy, DDPM and Transformer policies on real robot manipulation for evaluation. We carry out 50 replications of trials for each baseline. We use a KINOVA Gen3 arm and an Azure Kinect camera for real-world robot experiments. Details can be found in the supplementary video. Besides, we have respectively compared the proposed parameter-efficient affordance learning and flow matching policy against SoA baselines. In this section, we also investigate whether our framework can seamlessly unify affordance learning and action generation.

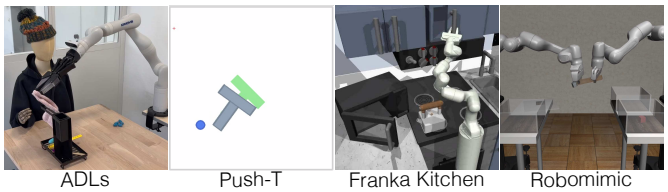
Affordance-based policy: From Table III, we can see that flow matching outperforms DDPM and Transformer baselines.

End-to-end learning (RGB images without affordances): We investigate how the affordance would guide the flow matching policy. We trained flow matching taking the raw RGB images and language tokens as input. This is similar to our proposed method but without the intermediate stage of affordance learning. It also involves some resemblance while not being entirely identical to the method in [4] in an end-to-end learning manner. Interestingly, comprehensive examinations reinforce the argument that flow matching could handle multimodal action distribution. Fig. 7 shows one example. From the left figure, we can see that when training a policy without affordances, the predicted trajectory (yellow) of moving the towel toward the trash for sweeping could be detached from the ground truth (red), but still a reasonable solution that allows for a successful robot execution. With affordance guidance, the prediction is closely aligned with the truth (Fig. 7-right, Table). We also observe that for applications that demand higher precision in manipulation, like grasping the toothbrush handle, affordances offer greater guidance for shaping the manipulation policy.

Off-the-shelf VLM-based trajectory generation: Our method uses an extra low-level module (flow matching) to learn a robot policy built on the high-level VLM reasoning. We have also conducted preliminary research to test another popular line of research, which uses a VLM to generate robot actions directly, as customary in the literature [27, 64]. No constant performance was achieved without carefully designed prompts or finetuning. This result is expected as several recent state-of-the-art research (e.g., HAMSTER [32], $\pi 0$ [4]) have shown that VLA models with robot learning modules outperform VLM-based trajectory generation.

D. Comparisons between flow matching and diffusion policy

To further investigate the performance of flow matching compared to diffusion policy, we benchmark the proposed methods on three more datasets which include closed-loop 6D robot actions and gripper actions: (i) Push-T [11], (ii) Franka Kitchen [18], and (iii) Robomimic [43].



Methods (16-step)	Push-T[a] ↑	Push-T[b] ↑	Franka Kitchen ↑	Robomimic ↑
Flow Matching	**0.9035/0.7519**	0.7363/**0.6218**	**0.9960**/0.7425	0.9360/**0.7289**
DDPM	0.8840/0.7178	0.7360/0.6100	0.9840/0.6716	0.9359/0.7168
DDIM	0.8801/0.6372	0.7490/0.6167	0.9865/**0.7471**	0.9334/0.7073
Score-based	0.8950/0.6432	**0.7659**/0.6186	0.9863/0.6892	**0.9470**/0.4896

[a] sampling range: [(50, 450), (50, 450), (200, 300), (200, 300), $(-\pi, \pi)$]
[b] sampling range: [(50, 450), (50, 450), (100, 400), (100, 400), $(-\pi, \pi)$]

TABLE IV: We present the robot evaluation performance in the format of (max performance) / (average of last checkpoint with 10 trials of replications), with each averaged across 500 different environment initial conditions. The metric used here is success rate, except for the Push-T task which uses target area coverage. We have used various sampling ranges (end-effector position, T-block position and orientation) for Push-T environment initialization. For Robomimic benchmark, we specifically report results on the Transport task.

- *Push-T* requires pushing a T-shaped block to a fixed target with a circular end-effector. Push-T takes RGB images with proprioception of end-effector location as inputs, and outputs end-effector actions in a closed-loop manner. The dataset includes 200 demonstrations.
- *Franka Kitchen* contains 7 objects for interaction and comes with a human demonstration dataset of 566 demonstrations, each completing 4 tasks in arbitrary order. The goal is to execute as many demonstrated tasks as possible, regardless of order. The training takes state-based inputs and outputs closed-loop robot joint actions and gripper actions.
- *Robomimic* consists of 5 tasks with a proficient human teleoperated demonstration dataset. We specifically focus on the transport task which includes 200 demonstrations. The policy takes state-based inputs and outputs closed-loop robot joint actions and gripper actions.

For each benchmark, the evaluation has been carried out across 500 different environment initial conditions, using the last checkpoint of each policy with 10 trials of replications. Thus, $5,000$ trials have been carried out in total per policy and benchmark. Variation is added on random initial conditions for the robot and object states. We respectively report the best and average performance in the 10 trials of replications of the last checkpoint. All state-based tasks are trained for $4,500$ epochs, and image-based tasks for $3,000$ epochs.

[7] has reported that setting prediction and action horizons greater than 1 helps the policy generate more consistent actions and compensate for idle portions of demonstrations; however, excessively long horizons can degrade performance due to slower reaction times. To ensure fair comparisons, we have selected the same corresponding hyperparameters across the flow matching policy and baseline methods to remain consistent. Table V shows the hyperparameters we have used in flow matching and diffusion policy. Table VI shows the task summary.

Similar conclusions could be achieved from Table IV and Fig. 8, as in the Main Results section:

- **Generation quality:** The performances of flow matching and DDPM, DDIM and the score-based model are comparable, where flow matching performs marginally better in most cases. We observe that score-based models can achieve higher peak performance, but their average performance tends to be less consistent.
- **Inference time:** Fig. 8 shows how the number of inference steps affects the performance of flow matching and diffusion policy. We can observe that diffusion policy showcases better performances when applying more inference iterations with a trade-off of longer inference time, as it requires a series of discrete steps to progressively refine the generated sample (solid line). Score-based models with black-box numerical ODE samplers, as in the original research paper [61], need adaptive steps to achieve optimal results (around $100-300$). Contrarily, flow matching has not shown significant improvements when increasing inference steps. From 8 steps onward, the performance of both flow matching and the diffusion policy increases only slightly with additional steps. However, the inference time (dotted line, we use frequency here) increases proportionally with the number of inference steps (dotted line). Therefore, flow matching considerably reduces the inference time for closed-loop robot manipulation. In the Push-T benchmark in Fig. 8, 2-step flow matching (coverage: 0.8803, time: 13.098ms) has achieved comparable performance as 16-step diffusion policy with DDIM (coverage: 0.8801, time: 98.268ms), **but noticeably lower inference time roughly by 86%**.

Based on the above experimental observations, we conclude that flow matching formulates the robot policy generation problem as learning a deterministic or near-deterministic vector field (via an ordinary differential equation) that maps noise (or a simple prior) to target action distributions—this ODE-based framing avoids iterative stochastic denoising chains (as in diffusion policy) and yields fast, single- or few-step inference, which is crucial in closed-loop robotic settings. Besides, in manipulation tasks, the target action distribution is often multimodal (e.g., multiple possible grasp poses, multiple contact sequences) but still has strong structure (e.g., physics constraints, smooth trajectories through state-action space). Flow matching's vector-field representation permits modeling such multimodality while preserving smooth interpolation and strong inductive bias towards consistent flows—thus potentially better capturing the structured trajectory manifold than, say, a GAN which may collapse modes, or a diffusion model which requires many steps. Moreover, because robotic manipulation is extremely sensitive to inference latency, stability, and feedback responsiveness, flow matching's fewer-step generation is advantageous. The explicit mapping of a vector field lends itself more naturally to feedback correction and incorporation of geometric/physical structure (e.g., SE(3) equivariance) than purely stochastic models. Overall, flow matching combines fast inference and structured multimodal representation, making it a compelling choice for manipulation domains.

During our experiments, we observed that most failures were caused by out-of-distribution factors. In real-world experiments,

H-Param	Ta	Tp	ObsRes	F-Net	F-Par	V-Enc	V-Par	Lr	WDe	Iters Train (FM)	Iters Train (DDPM)	Iters Train (DDIM)	Iters Eval
Activities of Daily Living	32	32	1x224x224	ConditionalUnet1D	72	ResNet-18	11	1e-4	1e-6	N/A	1/4/8/16	16	1/4/8/16
Push-T	8	16	1x96x96	ConditionalUnet1D	80	ResNet-18	11	1e-4	1e-6	N/A	1/4/8/16	16	1/4/8/16
Franka Kitchen	8	16	1x60	ConditionalUnet1D	66	N/A	N/A	1e-4	1e-6	N/A	1/4/8/16	16	1/4/8/16
Robomimic	8	16	1x50	ConditionalUnet1D	66	N/A	N/A	1e-4	1e-6	N/A	1/4/8/16	16	1/4/8/16

TABLE V: Hyperparameters for flow matching and diffusion policy. Ta: action horizon. Tp: action prediction horizon. ObsRes: environment observation resolution. D-Net: diffusion/flow matching network. D-Par: diffusion/flow matching network number of parameters in millions. V-Enc: vision encoder. V-Par: vision encoder number of parameters in millions. Lr: learning rate. WDe: weight decay. Iters Train: number of training diffusion iterations. Iters Eval: number of inference iterations.

Tasks	Rob	Obj	ActD	PH	Steps	Img	Closed-loop
Activities of Daily Living	1	≈ 30	3	8,000	N/A	✓	✗
Push-T	1	1	2	200	300	✓	✓
Franka Kitchen	1	7	9	566	280	✗	✓
Robomimic	2	3	20	200	700	✗	✓

TABLE VI: Tasks Summary. Rob: number of robots. Obj: number of objects. ActD: action dimension. PH: proficient-human demonstration. Steps: max number of rollout steps. Franka Kitchen and Robomimic involve 6D robot and gripper actions in the joint space. ADLs and Push-T focus on robot end-effector trajectories. For clarity, we further explain that for our ADLs tasks, as no closed-loop motion is considered here, we assume that the gripper closes when the first waypoint has arrived, and the low-level actions are executed between waypoints using a standard proportional-derivative (PD) controller.

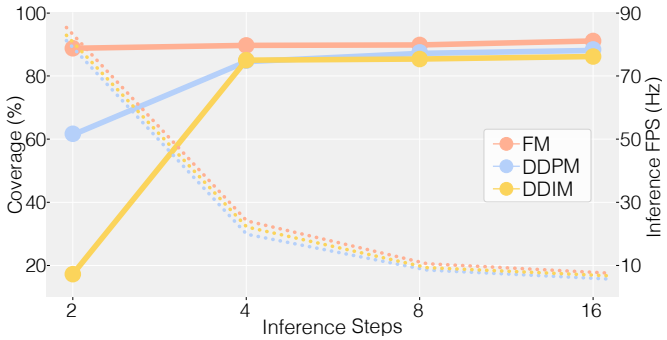


Fig. 8: Generation quality (solid line) and inference time (dotted line, we use frequency here) comparison of flow matching and diffusion policy for varying values of inference steps.

variations such as background changes or significant camera view shifts often led to failures of the 3D flow-matching policy. Increasing both the dataset size and diversity could help mitigate this issue. Unlike the simulation benchmarks, we performed open-loop manipulation in the real world, meaning that a failed first grasp attempt was counted as a failure; closed-loop manipulation could potentially address this limitation. We also found that tasks requiring higher manipulation precision, such as grasping a toothbrush handle, exhibited a higher probability of failure. In simulation benchmarks, out-of-distribution failures were also observed. For example, in the Push-T benchmark, only successful trajectories were included during training, so the policy struggled with edge cases, such as a T-block stuck in a corner. Extending the imitation policy to an online or offline reinforcement learning framework may help address these limitations.

V. Limitations

In this section, we provide a detailed analysis of the advantages and limitations of our proposed affordance-based flow matching approach.

Inference time: The generation quality of flow matching and diffusion policy for robot manipulation are generally comparable. Although we see only a marginal improvement in flow matching in most cases, we would like to highlight that the focus of this work is not to outperform state-of-the-art general robot manipulation research. Instead, we have systematically studied the flow matching framework, which provides an alternative to diffusion policies for robot manipulation. We can not overlook the additional advantages of flow matching, including stable training, easy implementation, and most importantly, significantly better performance and faster inference with fewer inference steps than diffusion policy, suggesting forsaking the stochastic construction of diffusion policy in favor of learning the probability path more directly as in flow matching.

We have evaluated how varying inference steps affect the performance of flow matching and diffusion policy. The original diffusion policy research [7] uses a large 100 training and inference steps with DDPM in their experiments. Based on our evaluation in Fig. 8 and results from other research [8], we can observe that beyond 8 steps, further increasing steps have only a marginal impact on the performance of diffusion policy, but with a trade-off of significantly longer inference time. We are aware of recent research on one-step diffusion policy with distillation [51] and shortcut models [13]. We primarily focus on conducting a comparative analysis of the fundamental architectures underlying flow matching and vanilla diffusion policy with DDPM, DDIM and the score-based model.

Deterministic and stochastic tasks: Flow Matching is inherently more deterministic than diffusion-based approaches, which can have implications in highly stochastic tasks—i.e., tasks where the same actions can lead to different outcomes due to environmental randomness or uncertainty, such as manipulating deformable objects, pushing objects on uneven surfaces, or navigating in dynamic environments. The impact of flow matching's determinism also depends on the training dataset: with a diverse dataset covering a wide range of outcomes, flow matching can learn to predict the most likely trajectories effectively, whereas in limited or biased datasets, flow matching may fail to capture rare but plausible outcomes.

Trajectory smoothness: Although our flow matching framework effectively learns Cartesian-space or joint-space trajectory representations, it does not explicitly enforce second-order

smoothness or torque feasibility during training or inference. In practice, the trajectories are executed through standard robot APIs (e.g., Kinova), which apply spline-based interpolation and internal velocity/acceleration limits to ensure dynamically feasible and smooth motion. Nevertheless, incorporating explicit smoothness or dynamic-feasibility constraints—such as velocity or acceleration regularization terms, or post-processing filters—could further improve motion consistency and physical plausibility. However, as noted in recent works [55, 72], these additions typically increase computational complexity, highlighting a trade-off between control fidelity and real-time efficiency. Exploring this direction is a valuable avenue for future research.

Transportation cost: While our method successfully leverages basic flow matching to generate robust and feasible affordance-based trajectories with a high success rate, we acknowledge that it does not explicitly pursue the minimal geometric transportation cost. More advanced flow techniques, such as Rectified Flow Matching (RFM) [37] or methods based on Optimal Transport (OT) [66], are mathematically designed to produce straight-line mappings that minimize this cost. However, implementing these methods introduces significant training and preprocessing overhead, often necessitating complex iterative or global optimization schemes. Given that our primary goal is to explore the fundamental structural efficacy and computational tractability and use affordances to shape the target distribution, we adopt the more efficient basic flow matching objective. We are deferring the complex, multi-objective optimization that simultaneously constrains geometric factors like path length and crucial practical constraints to future work.

Scalability: The current scope of our work presents several avenues for future generalization. The ADLs dataset was constructed in a controlled setting. We acknowledge this limited variation necessitates future work in demonstrating cross-domain generalization (e.g., varying lighting or camera poses), as acquiring and annotating the vast, diverse datasets required for open-world robustness is often infeasible for typical research settings. Furthermore, while our parameter-efficient prompt tuning leverages the semantic robustness of a pretrained foundation model to learn manipulation affordances, incorporating the spatial and semantic interactions between elements in multi-task scenarios, its ultimate performance against extreme linguistic noise is fundamentally data-driven. We plan to address these challenges in future work by expanding dataset diversity and evaluation rigor. Besides, we are exploring future extensions that incorporate additional modalities (e.g., haptic or force-torque sensing) or dynamics models to provide a richer, dynamics-aware affordance representation.

VI. Conclusion

We have formulated a prompt tuning method for affordance map learning and flow matching policy for robot manipulation. The core idea of prompt tuning is to maximally exploit the pretrained foundation model, and rapidly excavate the relevance of foundation and downstream affordance learning tasks. We have proposed a flow matching policy constructing paths that allow faster inference, and improved generation amongst robot behavior cloning methods. We qualitatively and quantitatively experiment on multiple robot manipulation benchmarks to prove that flow matching produces better trade-offs between computational cost and sample quality compared to prior competing diffusion-based methods.

References

[1] Michael S Albergo and Eric Vanden-Eijnden. Building normalizing flows with stochastic interpolants. *arXiv preprint arXiv:2209.15571*, 2022.

[2] Shikhar Bahl, Russell Mendonca, Lili Chen, Unnat Jain, and Deepak Pathak. Affordances from human videos as a versatile representation for robotics. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 13778–13790, 2023.

[3] Homanga Bharadhwaj, Roozbeh Mottaghi, Abhinav Gupta, and Shubham Tulsiani. Track2act: Predicting point tracks from internet videos enables diverse zero-shot robot manipulation. *arXiv e-prints*, pages arXiv–2405, 2024.

[4] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.

[5] Denis Blessing, Onur Celik, Xiaogang Jia, Moritz Reuss, Maximilian Li, Rudolf Lioutikov, and Gerhard Neumann. Information maximizing curriculum: A curriculum-based approach for learning versatile skills. *Advances in Neural Information Processing Systems*, 36:51536–51561, 2023.

[6] Max Braun, Noémie Jaquier, Leonel Rozo, and Tamim Asfour. Riemannian flow matching policy for robot motion learning. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 5144–5151. IEEE, 2024.

[7] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *arXiv preprint arXiv:2303.04137*, 2023.

[8] Eugenio Chisari, Nick Heppert, Max Argus, Tim Welschehold, Thomas Brox, and Abhinav Valada. Learning robotic manipulation policies from point clouds with conditional flow matching. *arXiv preprint arXiv:2409.07343*, 2024.

[9] Thanh-Toan Do, Anh Nguyen, and Ian Reid. Affordancenet: An end-to-end deep learning approach for object affordance detection. In *2018 IEEE international conference on robotics and automation (ICRA)*, pages 5882–5889. IEEE, 2018.

[10] Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, Harry Saini, Yam Levi, Dominik Lorenz, Axel Sauer, Frederic Boesel, et al. Scaling rectified flow transformers for high-resolution image synthesis. In *Forty-first International Conference on Machine Learning*, 2024.

[11] Pete Florence, Corey Lynch, Andy Zeng, Oscar A Ramirez, Ayzaan Wahid, Laura Downs, Adrian Wong,

Johnny Lee, Igor Mordatch, and Jonathan Tompson. Implicit behavioral cloning. In *Conference on Robot Learning*, pages 158–168. PMLR, 2022.

[12] Peter R Florence, Lucas Manuelli, and Russ Tedrake. Dense object nets: Learning dense visual object descriptors by and for robotic manipulation. *arXiv preprint arXiv:1806.08756*, 2018.

[13] Kevin Frans, Danijar Hafner, Sergey Levine, and Pieter Abbeel. One step diffusion via shortcut models. *arXiv preprint arXiv:2410.12557*, 2024.

[14] Roy Ganz, Yair Kittenplon, Aviad Aberdam, Elad Ben Avraham, Oren Nuriel, Shai Mazor, and Ron Litman. Question aware vision transformer for multimodal reasoning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 13861–13871, 2024.

[15] Peng Gao, Shijie Geng, Renrui Zhang, Teli Ma, Rongyao Fang, Yongfeng Zhang, Hongsheng Li, and Yu Qiao. Clip-adapter: Better vision-language models with feature adapters. *International Journal of Computer Vision*, 132 (2):581–595, 2024.

[16] James J Gibson. *The ecological approach to visual perception: classic edition*. Psychology press, 2014.

[17] Ankit Goyal, Jie Xu, Yijie Guo, Valts Blukis, Yu-Wei Chao, and Dieter Fox. Rvt: Robotic view transformer for 3d object manipulation. In *Conference on Robot Learning*, pages 694–710. PMLR, 2023.

[18] Abhishek Gupta, Vikash Kumar, Corey Lynch, Sergey Levine, and Karol Hausman. Relay policy learning: Solving long-horizon tasks via imitation and reinforcement learning. *arXiv preprint arXiv:1910.11956*, 2019.

[19] Tanmay Gupta, Amita Kamath, Aniruddha Kembhavi, and Derek Hoiem. Towards general purpose vision systems: An end-to-end task-agnostic vision-language architecture. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 16399–16409, 2022.

[20] Junxian He, Chunting Zhou, Xuezhe Ma, Taylor Berg-Kirkpatrick, and Graham Neubig. Towards a unified view of parameter-efficient transfer learning. *arXiv preprint arXiv:2110.04366*, 2021.

[21] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.

[22] Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross Girshick. Masked autoencoders are scalable vision learners. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 16000–16009, 2022.

[23] Jonathan Ho and Stefano Ermon. Generative adversarial imitation learning. *Advances in neural information processing systems*, 29, 2016.

[24] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020.

[25] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.

[26] Xixi Hu, Bo Liu, Xingchao Liu, and Qiang Liu. Adaflow: Imitation learning with variance-adaptive flow-based policies. *arXiv preprint arXiv:2402.04292*, 2024.

[27] Wenlong Huang, Chen Wang, Ruohan Zhang, Yunzhu Li, Jiajun Wu, and Li Fei-Fei. Voxposer: Composable 3d value maps for robotic manipulation with language models. *arXiv preprint arXiv:2307.05973*, 2023.

[28] Nils Ingelhag, Jesper Munkeby, Jonne van Haastregt, Anastasia Varava, Michael C Welle, and Danica Kragic. A robotic skill learning system built upon diffusion policies and foundation models. *arXiv preprint arXiv:2403.16730*, 2024.

[29] Menglin Jia, Luming Tang, Bor-Chun Chen, Claire Cardie, Serge Belongie, Bharath Hariharan, and Ser-Nam Lim. Visual prompt tuning. In *European Conference on Computer Vision*, pages 709–727. Springer, 2022.

[30] Yunfan Jiang, Agrim Gupta, Zichen Zhang, Guanzhi Wang, Yongqiang Dou, Yanjun Chen, Li Fei-Fei, Anima Anandkumar, Yuke Zhu, and Linxi Fan. Vima: General robot manipulation with multimodal prompts. *arXiv preprint arXiv:2210.03094*, 2(3):6, 2022.

[31] Sergey Levine, Chelsea Finn, Trevor Darrell, and Pieter Abbeel. End-to-end training of deep visuomotor policies. *Journal of Machine Learning Research*, 17(39):1–40, 2016.

[32] Yi Li, Yuquan Deng, Jesse Zhang, Joel Jang, Marius Memmel, Raymond Yu, Caelan Reed Garrett, Fabio Ramos, Dieter Fox, Anqi Li, et al. Hamster: Hierarchical action models for open-world robot manipulation. *arXiv preprint arXiv:2502.05485*, 2025.

[33] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as policies: Language model programs for embodied control. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pages 9493–9500. IEEE, 2023.

[34] Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*, 2022.

[35] Fangchen Liu, Kuan Fang, Pieter Abbeel, and Sergey Levine. Moka: Open-vocabulary robotic manipulation through mark-based visual prompting. *arXiv preprint arXiv:2403.03174*, 2024.

[36] Pengfei Liu, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *ACM Computing Surveys*, 55(9):1–35, 2023.

[37] Qiang Liu. Rectified flow: A marginal preserving approach to optimal transport. *arXiv preprint arXiv:2209.14577*, 2022.

[38] Xiao Liu, Kaixuan Ji, Yicheng Fu, Weng Lam Tam, Zhengxiao Du, Zhilin Yang, and Jie Tang. P-tuning v2: Prompt tuning can be comparable to fine-tuning universally across scales and tasks. *arXiv preprint arXiv:2110.07602*, 2021.

[39] Zuxin Liu, Jesse Zhang, Kavosh Asadi, Yao Liu, Ding Zhao, Shoham Sabach, and Rasool Fakoor. Tail: Task-specific adapters for imitation learning with large pre-trained models. *arXiv preprint arXiv:2310.05905*, 2023.

[40] Cheng Lu and Yang Song. Simplifying, stabilizing and scaling continuous-time consistency models. *arXiv preprint arXiv:2410.11081*, 2024.

[41] Yecheng Jason Ma, William Liang, Guanzhi Wang, De-An Huang, Osbert Bastani, Dinesh Jayaraman, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Eureka: Human-level reward design via coding large language models. *arXiv preprint arXiv:2310.12931*, 2023.

[42] Martin Maier and Rasha Abdel Rahman. No matter how: Top-down effects of verbal and semantic category knowledge on early visual perception. *Cognitive, Affective, & Behavioral Neuroscience*, 19:859–876, 2019.

[43] Ajay Mandlekar, Danfei Xu, Josiah Wong, Soroush Nasiriany, Chen Wang, Rohun Kulkarni, Li Fei-Fei, Silvio Savarese, Yuke Zhu, and Roberto Martín-Martín. What matters in learning from offline human demonstrations for robot manipulation. In *arXiv preprint arXiv:2108.03298*, 2021.

[44] Lucas Manuelli, Wei Gao, Peter Florence, and Russ Tedrake. kpam: Keypoint affordances for category-level robotic manipulation. In *The International Symposium of Robotics Research*, pages 132–157. Springer, 2019.

[45] Preetum Nakkiran, Arwen Bradley, Hattie Zhou, and Madhu Advani. Step-by-step diffusion: An elementary tutorial. *arXiv preprint arXiv:2406.08929*, 2024.

[46] Soroush Nasiriany, Sean Kirmani, Tianli Ding, Laura Smith, Yuke Zhu, Danny Driess, Dorsa Sadigh, and Ted Xiao. Rt-affordance: Affordances are versatile intermediate representations for robot manipulation. *arXiv preprint arXiv:2411.02704*, 2024.

[47] Khang Nguyen, An T Le, Tien Pham, Manfred Huber, Jan Peters, and Minh Nhat Vu. Flowmp: Learning motion fields for robot planning with conditional flow matching. *arXiv preprint arXiv:2503.06135*, 2025.

[48] Abhishek Padalkar, Acorn Pooley, Ajinkya Jain, Alex Bewley, Alex Herzog, Alex Irpan, Alexander Khazatsky, Anant Rai, Anikait Singh, Anthony Brohan, et al. Open x-embodiment: Robotic learning datasets and rt-x models. *arXiv preprint arXiv:2310.08864*, 2023.

[49] Ethan Perez, Florian Strub, Harm De Vries, Vincent Dumoulin, and Aaron Courville. Film: Visual reasoning with a general conditioning layer. In *Proceedings of the AAAI conference on artificial intelligence*, volume 32, 2018.

[50] Gabriel Peyré, Marco Cuturi, et al. Computational optimal transport: With applications to data science. *Foundations and Trends® in Machine Learning*, 11(5-6):355–607, 2019.

[51] Aaditya Prasad, Kevin Lin, Jimmy Wu, Linqi Zhou, and Jeannette Bohg. Consistency policy: Accelerated visuomotor policies via consistency distillation. *arXiv preprint arXiv:2405.07503*, 2024.

[52] Shengyi Qian, Weifeng Chen, Min Bai, Xiong Zhou, Zhuowen Tu, and Li Erran Li. Affordancellm: Grounding affordance from vision language models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 7587–7597, 2024.

[53] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pages 8748–8763. PMLR, 2021.

[54] Moritz Reuss, Maximilian Li, Xiaogang Jia, and Rudolf Lioutikov. Goal-conditioned imitation learning using score-based diffusion policies. *arXiv preprint arXiv:2304.02532*, 2023.

[55] Ralf Römer, Alexander von Rohr, and Angela P Schoellig. Diffusion predictive control with constraints. *arXiv preprint arXiv:2412.09342*, 2024.

[56] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In *Medical image computing and computer-assisted intervention–MICCAI 2015: 18th international conference, Munich, Germany, October 5-9, 2015, proceedings, part III 18*, pages 234–241. Springer, 2015.

[57] Quentin Rouxel, Andrea Ferrari, Serena Ivaldi, and Jean-Baptiste Mouret. Flow matching imitation learning for multi-support manipulation. In *2024 IEEE-RAS 23rd International Conference on Humanoid Robots (Humanoids)*, pages 528–535. IEEE, 2024.

[58] Mohit Sharma, Claudio Fantacci, Yuxiang Zhou, Skanda Koppula, Nicolas Heess, Jon Scholz, and Yusuf Aytar. Lossless adaptation of pretrained vision models for robotic manipulation. *arXiv preprint arXiv:2304.06600*, 2023.

[59] Kihyuk Sohn, Huiwen Chang, José Lezama, Luisa Polania, Han Zhang, Yuan Hao, Irfan Essa, and Lu Jiang. Visual prompt tuning for generative transfer learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 19840–19851, 2023.

[60] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. *arXiv preprint arXiv:2010.02502*, 2020.

[61] Yang Song, Jascha Sohl-Dickstein, Diederik P Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. *arXiv preprint arXiv:2011.13456*, 2020.

[62] Kaustubh Sridhar, Souradeep Dutta, Dinesh Jayaraman, James Weimer, and Insup Lee. Memory-consistent neural networks for imitation learning. *arXiv preprint arXiv:2310.06171*, 2023.

[63] Austin Stone, Ted Xiao, Yao Lu, Keerthana Gopalakrishnan, Kuang-Huei Lee, Quan Vuong, Paul Wohlhart, Sean Kirmani, Brianna Zitkovich, Fei Xia, et al. Open-world object manipulation using pre-trained vision-language models. *arXiv preprint arXiv:2303.00905*, 2023.

[64] Daniel Tanneberg, Felix Ocker, Stephan Hasler, Joerg Deigmoeller, Anna Belardinelli, Chao Wang, Heiko Wersing, Bernhard Sendhoff, and Michael Gienger. To help or not to help: Llm-based attentive support for human-robot group interactions. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*,

pages 9130–9137. IEEE, 2024.

[65] Laurens Van der Maaten and Geoffrey Hinton. Visualizing data using t-sne. *Journal of machine learning research*, 9(11), 2008.

[66] Cédric Villani. *Topics in optimal transportation*, volume 58. American Mathematical Soc., 2021.

[67] Yen-Jen Wang, Bike Zhang, Jianyu Chen, and Koushil Sreenath. Prompt a robot to walk with large language models. *arXiv preprint arXiv:2309.09969*, 2023.

[68] Tete Xiao, Ilija Radosavovic, Trevor Darrell, and Jitendra Malik. Masked visual pre-training for motor control. *arXiv preprint arXiv:2203.06173*, 2022.

[69] Yigit Yildirim and Emre Ugur. Conditional neural expert processes for learning movement primitives from demonstration. *IEEE Robotics and Automation Letters*, 2024.

[70] Wentao Yuan, Jiafei Duan, Valts Blukis, Wilbert Pumacay, Ranjay Krishna, Adithyavairavan Murali, Arsalan Mousavian, and Dieter Fox. Robopoint: A vision-language model for spatial affordance prediction for robotics. *arXiv preprint arXiv:2406.10721*, 2024.

[71] Andy Zeng, Pete Florence, Jonathan Tompson, Stefan Welker, Jonathan Chien, Maria Attarian, Travis Armstrong, Ivan Krasin, Dan Duong, Vikas Sindhwani, et al. Transporter networks: Rearranging the visual world for robotic manipulation. In *Conference on Robot Learning*, pages 726–747. PMLR, 2021.

[72] Wei Zeng, Tifan Xiong, and Chao Wang. Optimization of smooth trajectories for two-wheel differential robots under kinematic constraints using clothoid curves. *Sensors*, 25 (10):3143, 2025.

[73] Fan Zhang and Yiannis Demiris. Learning garment manipulation policies toward robot-assisted dressing. *Science robotics*, 7(65):eabm6010, 2022.

[74] Yun Zhong and Yiannis Demiris. Dancemvp: Self-supervised learning for multi-task primitive-based dance performance assessment via transformer text prompting. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 10270–10278, 2024.

[75] Yun Zhong, Fan Zhang, and Yiannis Demiris. Contrastive self-supervised learning for automated multi-modal dance performance assessment. In *ICASSP 2023-2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 1–5. IEEE, 2023.

[76] Jiawen Zhu, Simiao Lai, Xin Chen, Dong Wang, and Huchuan Lu. Visual prompt multi-modal tracking. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 9516–9526, 2023.